AstralMath-v1: A Multi-Model Tool-Integrated Reasoning Dataset for Competition Mathematics

Nguyen Nguyen
Independent Researcher

Abstract

We introduce ASTRALMATH-V1, a large-scale dataset of 431k+ Stateful Tool-Integrated Reasoning (TIR) solution traces for 136,151 competition-level mathematics problems with over 3.2B-token. We provide a large number of inference traces from 7 frontier open-source models, generating diverse reasoning strategies and code patterns, serving as input for model fine-tuning, knowledge extraction, or reinforcement learning. A two-phase synthetic transformation pipeline converts problems into verifiable numeric answer variants and verifies every transformed answer through 12-run consensus. We also release ASTRALMATH-V1-ERRORTRACES, a companion corpus of 520k+ failure traces enabling systematic study of model failure modes and preference-based optimization such as Direct Preference Optimization (DPO). For evaluation, we curate ASTRALBENCH, a 50-problem benchmark drawn from elite competition sources, where current frontier models achieve 15–30% accuracy. The dataset, benchmark, and error traces are publicly available on HuggingFace.

🤗 **Dataset:** https://huggingface.co/collections/nguyen599/astral-dataset

⭐ **Vizual:** https://nguyen599.github.io/aimo3-astral/

1 Introduction

Mathematical reasoning has emerged as one of the most challenging and actively studied capabilities of large language models (LLMs) [Hendrycks et al., 2021, Cobbe et al., 2021]. Recent advances in reinforcement learning with verifiable rewards (RLVR) [Shao et al., 2024, Guo et al., 2025] have demonstrated that training on problems with deterministic, checkable answers produces substantial improvements in reasoning ability. Tool-Integrated Reasoning (TIR)—where models interleave natural language chain-of-thought with executable Python code—has proven particularly effective, enabling models to offload precise computation to a code interpreter while focusing their reasoning on problem decomposition and strategy selection [Gou et al., 2024]. These developments have made three properties essential for modern mathematical training data: verifiable answers suitable for RLVR, sufficient difficulty that challenges current frontier models, and rich TIR traces with real tool-call outputs.

Despite a growing landscape of open-source mathematical datasets, existing resources address these requirements only partially. Large-scale corpora such as NuminaMath [Li et al., 2024a] ($\sim$900K problems) and OpenMathInstruct-2 [Toshniwal et al., 2024a] (14M solution pairs) provide impressive scale but focus on chain-of-thought solutions without tool use, limiting their applicability for TIR training. Nemotron-Math [Du et al., 2025] offers 7.5M TIR traces with multiple reasoning modes, but all solutions originate from a single generator model family, producing uniform solution styles and code patterns. DeepMath-103K [He et al., 2025] prioritises rigorous decontamination and verification

Preprint.

but provides only 103K problems without tool-use traces. ToRA [Gou et al., 2024] was the first to demonstrate the value of tool-integrated reasoning for mathematical problem solving, yet its dataset targets GSM8K/MATH-level difficulty—problems that frontier models now solve routinely. This suggests an opportunity for a dataset that jointly emphasizes competition-level difficulty, multi-model TIR traces, and robust verification suitable for both SFT and RLVR training.

These observations suggest several opportunities for a dataset that better aligns with the needs of modern mathematical reasoning research. In particular, an ideal resource would combine diverse solution traces from multiple model families, include a substantial collection of error traces for failure analysis, and support benchmark evaluation at competition level with verifiable integer answers. Motivated by these goals, we introduce ASTRALMATH-V1, a comprehensive multi-model TIR dataset for competitive mathematics built from the diversity of today's frontier reasoning models. Our contributions are as follows:

1. We release **ASTRALMATH-V1**, 431,861 long-form solution trajectories from seven frontier models (DeepSeek-V3.2, GPT-OSS-120B, Step-3.5-Flash, GLM-5, Kimi-K2.5, Qwen3.5-397B-A17B, and Nemotron-3-Super) using Stateful Python TIR with over 3.2B tokens. Every datapoint carries rich metadata including success rate, token length, per-tool execution times, and libraries used, enabling difficulty-aware curriculum construction.
2. We applied **consensus-verified synthetic transformations**: a two-phase pipeline that converts problems with symbolic or multi-valued answers into competition-style integer-answer questions. A planner selects the transformation strategy; then 12 independent verifier calls must *unanimously agree* on the transformed answer before acceptance, eliminating hallucinated arithmetic at scale.
3. We release **ASTRALMATH-V1-ERRORTRACES**, 520,000+ failure traces categorised into six error types (incorrect answer, limit exceeded, unextractable output, and others), enabling diagnostic analysis of model failure modes and supporting preference-based optimisation methods such as DPO.
4. We curate **AstralBench**, a 50-problem benchmark from ten competition sources (IMO, EGMO, Putnam, HMMT, USA-TSTST, and others), with 28 problems manually transformed to integer-answer format; the GPT-OSS-120B model achieves 15–30% accuracy.

2 Related Work

Mathematical reasoning benchmarks. GSM8K [Cobbe et al., 2021] and MATH [Hendrycks et al., 2021] have served as the primary evaluation targets for mathematical reasoning, but frontier models now achieve high accuracy on both, reducing their discriminative power for evaluating advanced reasoning. To address benchmark saturation, the community has developed increasingly difficult evaluations. IMO-Bench [Luong et al., 2025b] draws from International Mathematical Olympiad problems; Putnam-AXIOM [Gulati et al., 2025] targets university-level competition mathematics; Omni-MATH [Gao et al., 2024] provides Olympiad-level problems across diverse topics; and FrontierMath [Glazer et al., 2024] curates research-level problems where models currently score below 5%. Dynamic benchmarks have also emerged: MathArena [Balunovic et al., 2025] provides a continuously updated leaderboard from recent competition problems to reduce contamination risk, while RealMath [Zhang et al., 2025] refreshes its evaluation set from newly published arXiv papers and Mathematics Stack Exchange posts. ASTRALBENCH complements this landscape by targeting a calibrated difficulty range (15–30% frontier accuracy), positioned between the high-accuracy MATH/AIME tier and more challenging benchmarks such as FrontierMath, while adopting an integer-answer evaluation format suitable for automated verification.

Math training datasets. The scale and quality of mathematical training data have grown rapidly. NuminaMath [Li et al., 2024a] provides ∼900K competition problems with chain-of-thought solutions, establishing one of the largest open-source collections. OpenMathInstruct-1 [Toshniwal et al., 2024b] introduced a code-interpreter solution format with 1.8M pairs, and its successor OpenMathInstruct-2 [Toshniwal et al., 2024a] scaled to 14M pairs while demonstrating that solution format, teacher model quality, and question diversity are key factors for effective training data. MetaMathQA [Yu et al., 2024] augments MATH and GSM8K problems through question rephrasing, showing that even simple augmentation can improve downstream performance. ScaleQuest [Ding et al., 2024] takes a different approach by synthesising entirely new problems from scratch using

Table 1: Feature comparison with existing math training datasets. † = Collected solutions and rewrites using GPT-4o. ‡ = Program-of-Thought (code execution without interleaved reasoning).

Dataset	Scale	Unique Qs	Generator model(s)	TIR
ASTRALMATH-V1 (ours)	431K	136K	7 models mixed	✓
OpenMathReasoning [Moshkov et al., 2025]	1.7M	540K	QwQ-32B	✓
MAmmoTH2 [Yue et al., 2024b]	8M	8M	Web-collected	–
OpenMathInstruct-2 [Toshniwal et al., 2024a]	14M	600K	Llama-3.1-405B	–
OpenMathInstruct-1 [Toshniwal et al., 2024b]	1.8M	∼60K	Mixtral	✓
Nemotron-Math [Du et al., 2025]	7.5M	347K	gpt-oss-120b	✓
NuminaMath [Li et al., 2024a]	860K	860K	GPT-4o †	–
NuminaMath-TIR [Li et al., 2024b]	70K	70K	GPT-4	✓
DART-Math [Tong et al., 2024]	591K	∼15K	DeepSeekMath-7B-RL	–
OpenR1-Math-220k [HuggingFace Open-R1 Team, 2025]	220K	220K	DeepSeek-R1	–
DeepMath-103K [He et al., 2025]	103K	103K	DeepSeek-R1	–
AceMath [Liu et al., 2024]	2.1M	–	Qwen2.5-Math-72B	–
ScaleQuest [Ding et al., 2024]	1M	1M	Qwen2-Math-7B	–
MAmmoTH [Yue et al., 2024a]	260K	∼100K	GPT-4 + GPT-3.5	‡
Orca-Math [Mitra et al., 2024]	200K	200K	GPT-4-Turbo	–
MetaMathQA [Yu et al., 2024]	395K	∼40K	GPT-3.5-Turbo	–

open-source models. DeepMath-103K [He et al., 2025] prioritises rigorous semantic decontamination against evaluation benchmarks and provides three R1-generated solutions per problem. Nemotron-Math [Du et al., 2025] generates 7.5M long-form reasoning traces under three controllable reasoning modes (high, medium, low) with and without Python TIR—but all traces originate from the gpt-oss-120b model, resulting in relatively homogeneous solution styles. OpenMathReasoning [Moshkov et al., 2025] provides 1.7M TIR solutions alongside 3.2M long reasoning CoT traces, generated from QwQ-32B with iterative training and quality filtering, the largest dedicated TIR component to date, primarily generated from a single model family. AceMath [Liu et al., 2024] demonstrates that high-quality post-training data combined with reward modelling can substantially advance frontier math performance. ASTRALMATH-V1 complements these efforts along two key axes: it provides multi-model TIR traces capturing the broader range reasoning strategies of seven different frontier models, and it releases a companion corpus of 520K+ error traces. Table 1 summarises the key feature differences.

Tool-integrated reasoning. ToRA [Gou et al., 2024] was the first to show that interleaving chain-of-thought reasoning with Python tool execution substantially improves performance on mathematical benchmarks, establishing the TIR paradigm. MAmmoTH [Yue et al., 2024a] extended hybrid instruction tuning to broader STEM domains. Subsequent work has focused on scaling TIR traces: Nemotron-Math [Du et al., 2025] provides 7.5M multi-mode traces and OpenMathReasoning [Moshkov et al., 2025] contributes 1.7M TIR solutions, both from single model families. ASTRALMATH-V1 explores a complementary direction for TIR data: in addition to scaling single-model outputs, we leverage multiple frontier models to produce traces with diverse code libraries, solution strategies, and debugging patterns for the same problems, at competition-level difficulty.

RLVR and curriculum learning for mathematics. DeepSeekMath [Shao et al., 2024] demonstrated that Group Relative Policy Optimisation (GRPO) with verifiable rewards substantially improves mathematical reasoning beyond SFT. DeepSeek-R1 [Guo et al., 2025] extended reinforcement learning to long-chain reasoning, producing models that rival human performance on competition mathematics. DAPO [Yu et al., 2025] provides an open-source RL framework for mathematical reasoning. Process reward models [Lightman et al., 2024] offer step-level supervision that can guide RL training more precisely than outcome-based rewards alone. Curriculum learning [Bengio et al., 2009] ordering training examples from easy to hard—has shown benefits in RL settings where the initial policy must occasionally succeed for learning to proceed. ASTRALMATH-V1 is designed to support these training paradigms by providing rich metadata and structured data splits: per-datapoint metadata (success rate, token length, tool execution times) enables difficulty-aware curriculum construction, while the the stage1/stage2 split provides a natural progression from initial supervised learning to more challenging RL-based training.

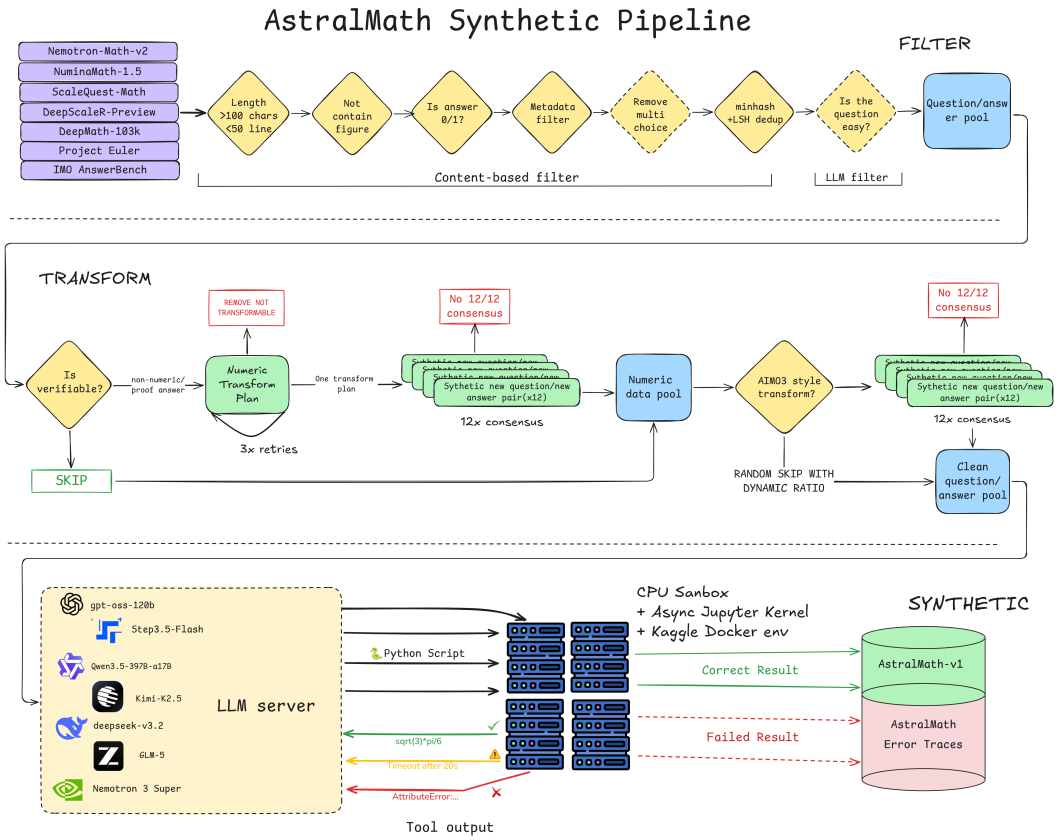


Figure 1: ASTRALMATH-V1 end-to-end pipeline. Dashed arrows indicate by-products: error traces are collected from failed TIR attempts to generate ASTRALMATH-V1-ERRORTRACES, and ASTRALBENCH is curated independently.

3 Dataset Construction

Figure 1 summarises the end-to-end pipeline.

3.1 Problem Sources

We aggregate problems from seven public sources chosen to maximise difficulty, topic diversity, and licence compatibility. Figure 2 shows the selection counts after filtering.

3.2 Multi-Stage Filtering

A critical design principle of ASTRALMATH-V1 is that, as model capabilities improve rapidly, easy questions provide diminishing training signal. We therefore apply a two-stage filtering pipeline—context-based heuristics followed by an LLM difficulty gate—to retain only problems that are hard yet solvable, reducing the initial $\sim$**2.3M** raw problems to **136,151** selected questions (6% retention).

Context-based filtering. The first stage removes problems that are unsuitable for TIR evaluation or unlikely to provide useful training signal. *Content filters* exclude questions shorter than 100 characters (typically underspecified) or exceeding 50 lines (excessive preamble), those containing image markers (`![]` / `figure` / `shown in the figure`), multiple-choice items (detected via pattern matching), and "double questions"—unrelated sub-questions bundled as one item, an artefact of the ScaleQuest corpus detected using an LLM judge. *Difficulty filters* remove problems with trivial ground-truth answers in $\{0, 1\}$ at a **70 %** exclusion rate, as models frequently default to these values when failing, producing false positives in RLVR training; this removes $\sim$6% of problems. Several source datasets provide per-problem metadata such as difficulty ratings and model pass rates; we exploit this metadata

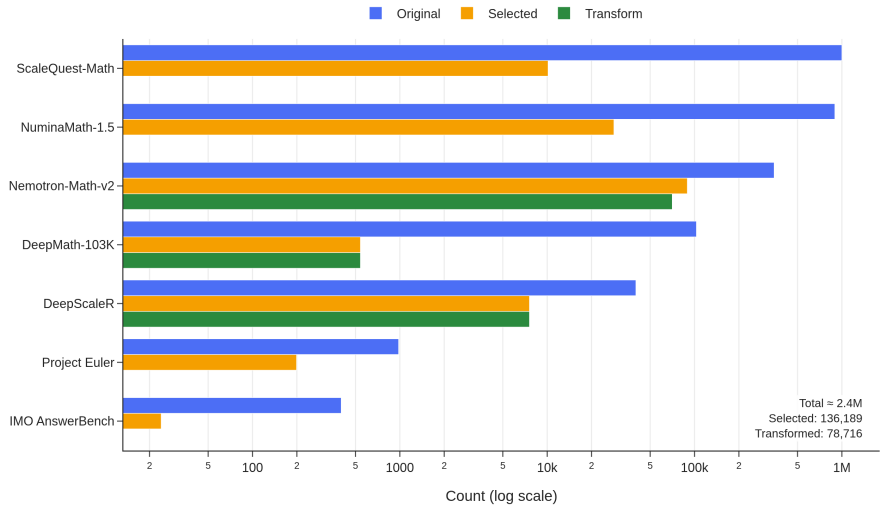
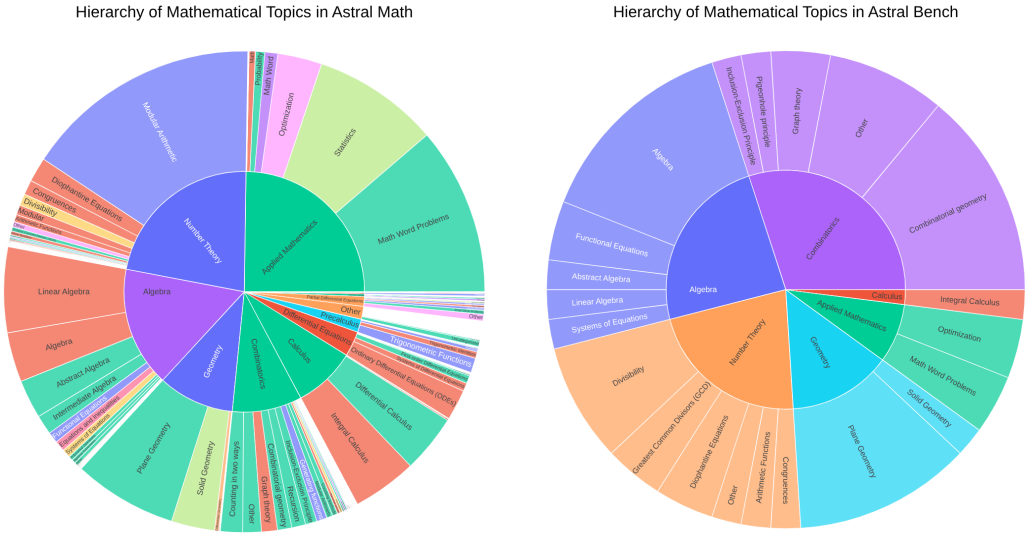


Figure 2: Problem sources and selection counts. "Transform" indicates the number of questions paired with a consensus-verified integer-answer transformation.



(a) ASTRALMATH-V1 (431K traces, 136K questions).

(b) ASTRALBENCH (50 problems).

Figure 3: Hierarchical topic distributions. ASTRALMATH-V1 (left) covers all major competition-mathematics areas broadly; ASTRALBENCH (right) concentrates on competition-core areas—Combinatorics, Algebra, Number Theory, and Geometry—reflecting its IMO-level sources.

to retain only the harder tail of each source (see Appendix A.1 for per-source details). *Deduplication* uses MinHash with Locality Sensitive Hashing (LSH) [Broder, 1997], identifying near-duplicate problems across the six heterogeneous source corpora and removing a further $\sim 2\%$.

LLM filtering. As a final difficulty gate, we generate a single no-tool solution for each remaining problem using `gpt-oss-120b` (high effort, max 28K tokens); any problem solved correctly on the first attempt is discarded. This stage is the most aggressive, eliminating $\sim 60\%$ of candidates, and ensures the retained problems resist easy one-shot solving.

Broad topical diversity. Each problem is classified into a hierarchical topic tree using an LLM judge. Figure 3a shows the resulting distribution: the dataset spans all major competition-mathematics

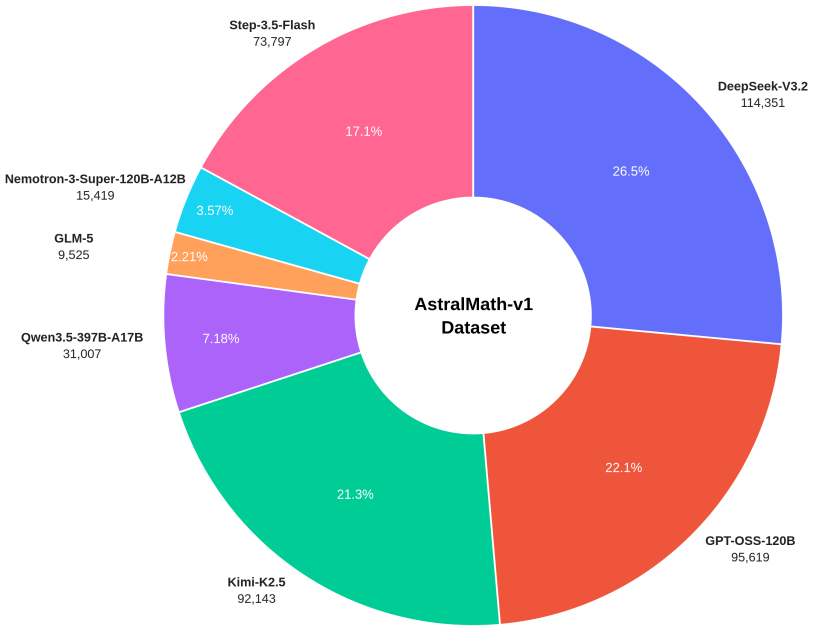


Figure 4: Distribution of Generated Traces by Model (Total N = 431,861).

areas, with Number Theory, Algebra, Applied Mathematics, Combinatorics, and Geometry as the dominant top-level categories. Within each area, coverage extends from fundamental topics (Prealgebra, Plane Geometry) to advanced domains (Abstract Algebra, Differential Equations), ensuring that models trained on ASTRALMATH-V1 encounter a rich variety of mathematical concepts and problem-solving paradigms. Figure 3b shows the corresponding ASTRALBENCH distribution, concentrated on competition-core areas consistent with its IMO-level sources.

3.3 Verifiable Numeric Transformations

Many problems have symbolic, set-valued, or multi-answer solutions that cannot be automatically verified. We convert these into numeric answer variants via a two-phase consensus pipeline: a *planner* LLM selects one of three transformation strategies (substitute parameters, aggregate, or not_transform) and produces a deterministic instruction to trasform question; then **N=12** independent *verifier* calls each solve the rewritten problem from scratch. A transformation is accepted *only if all 12 verifiers agree exactly*, ensuring high reliability. Full strategy descriptions, prompts, and complete examples are in Appendix B.

For problems whose answers are already numeric, a separate AIMO3 integer transform rewrites the question to request an integer answer and remainder modulo M, matching the AIMO3 competition format. We use a dynamic ratio to select a question that will transform to AIMO3 format to balance between dataset diversity and AIMO3-style evaluation. The AIMO3-style transformation is also verified with **12-run consensus**: a successful transformation replaces the original question with its integer-answer variant in the dataset; if verification fails, the original question is retained as-is. (detail at Appendix B.4).

3.4 Multi-Model TIR Generation

Phase 1: Library discovery. We first run a `gpt-oss-120b` and `Deepseek-V3.2` model with internet access on a 30K-question subset (∼75K traces) and record every `pip install` attempt. This surfaces non-standard libraries that models use for competition problems: `z3-solver`, `ortools`, `cvxpy`, `galois`, `shapely`, and others. All discovered packages are pre-installed in the sandbox.

Phase 2: Multi-model generation. With internet disabled, models generate TIR solutions independently in a Docker container replicating the Kaggle GPU notebook environment. Figure 4 shows per-model trace counts. For each model–question pair, after an initial correct solution is found, improvement iterations are run: subsequent solutions are kept only if they are both shorter and faster. We set max tool call is **200 turns** and each traces have maximum **97k** characters. Tool execution timeouts are sampled uniformly from **10–80 seconds** per question to encourage diverse solution strategies.

Execution environment. We used the official Kaggle Docker image and apply identical configuration. The Python sandbox is modified to support large-scale concurrent session creation and deletion, handling thousands of simultaneous Jupyter kernels with per-kernel **30GB** memory limits. Every tool-call output (including tracebacks) is recorded verbatim in the conversation trajectory.

4 Error Traces

Alongside the 431K correct solution traces, the TIR generation process produces a large volume of *failed* attempts. We release these as ASTRALMATH-V1-ERRORTRACES, comprising 520,000+ traces

Error taxonomy. The expected behaviors include:

1. **Incorrect answer**: model produces a final answer that does not match the ground truth, despite a coherent reasoning chain.
2. **Limit exceeded**: model reaches the maximum turn limit (200 turns) or maximum output length (97K characters) without finding a correct answer.
3. **Unextractable answer**: output is malformed or missing `\boxed{}` formatting, preventing answer extraction.
4. **Non-verifiable**: the verification function fails or produces inconsistent results.
5. **Incomplete question**: the source question is missing information (a data-quality issue), detected during generation.
6. **Ground-truth mismatch**: model output conflicts with the expected answer due to ambiguous or incorrect ground truth.

Evaluative role. Error traces enable three lines of investigation: (1) training error-detection and self-correction models; (2) studying model behaviors, how failure patterns differ systematically across models, like whether some models are more prone to formatting errors while others make reasoning mistakes; (3) constructing preference pairs (correct trace, error trace) for DPO-style training.

5 AstralBench

ASTRALBENCH is a 50-problem benchmark curated to support reproducible offline evaluation of mathematical reasoning models in a competition-style TIR setting, designed to simulate the AIMO3 leaderboard environment. All answers are integers in $[0, 99{,}999]$, enabling fully automated grading with no ambiguity.

Curation and sources. Problems are drawn from diverse competition sources, selected for difficulty and topic diversity. Table 2 shows the source breakdown. Question collected from IMO-AnswerBench, Project Euler, HMMT, SMT, USA-TSTST, USEMO, EGMO, CMO, Pumac, Putnam, Open-RL, and MIT-Math, to prevent source diversity and topic balance.

With problems that have symbolic or multi-valued answers in their original formulation, we manually transformed it by changing parameters, applying modular arithmetic (e.g., requesting remainder modulo a prime), and rephrasing the final question sentence, while preserving the core mathematical difficulty.

Difficulty calibration. ASTRALBENCH is designed to fill the gap between MATH/AIME-level benchmarks (now >90% for frontier models) and FrontierMath [Glazer et al., 2024] (where models score <5%). Table 3 shows scores spanning 6–30% across model scales: small 4B thinking models

Table 2: ASTRALBENCH source composition. "Transformed" indicates problems whose original answers were non-integer or symbolic and were manually converted.

Source	Count	Transformed
IMOBench	33	24
Project Euler	2	2
Open-RL	3	2
USA-TSTST	2	0
EGMO	2	0
Other (HMMT, SMT, USEMO, CMO, Pumac, Putnam, MIT-Math)	8	0
Total	**50**	**28**

Table 3: Baseline accuracy on ASTRALBENCH (Entropy-Weighted maj@8).

Model / Setting	Score	Accuracy
GPT-OSS-120B (5h, run 1)	12/50	24%
GPT-OSS-120B (5h, run 2)	15/50	30%
GPT-OSS-120B (10h)	13/50	26%
Qwen3-4B-Thinking (5h)	3/50	6%
DASD-4B-Thinking (5h)	5/50	10%
Qwen3-4B-Thinking-Finetune (5h)	9/50	18%

score 6–10%, fine-tuning on ASTRALMATH-V1 raises a 4B model to 18%, and the frontier GPT-OSS-120B reaches 24–30%. This spread demonstrates that ASTRALBENCH discriminates across both model scale and training regimen, providing meaningful headroom for development. All problems and a reference Kaggle evaluation notebook are publicly available, enabling direct comparison with the AIMO3 competition environment.

Baseline performance. Table 3 reports baseline accuracy on ASTRALBENCH using GPT-OSS-120B in the standard Kaggle notebook environment. Scores vary across runs due to stochastic generation and time limits.

6 Experiments

We present preliminary fine-tuning experiments to validate that ASTRALMATH-V1 provides useful training signal across model scales.

6.1 Experimental Setup

We fine-tune models from multiple families using either full SFT (for smaller models) or LoRA [Hu et al., 2022] (for larger models). Following the LoRA Without Regret methodology [Thinking Machines Lab, 2025], we use rank 1 with α=32, which has been shown to match full SFT performance at a fraction of the memory cost. Evaluation is performed on ASTRALBENCH v1.1. Table 4 summarises the training configuration for each model.

Table 4: Training hyperparameters for fine-tuning experiments.

	Qwen3-4B	GPT-OSS-20B	GPT-OSS-120B
Method	SFT	LoRA (r=1)	LoRA (r=1)
LoRA α	–	32	32
Steps	1,150	400	250
Max length	40,960	50,176	40,960
Batch size	8	12	16
Data	8K stage1	stage2	stage2

Table 5: Fine-tuning results on AIMO3 public leaderboard (maj@8).

Model	Original	Finetuned	Method	Data
Qwen3-4B [Qwen Team, 2025]	5/50	22–23/50	SFT	8K stage1
DASD-4B-Thinking [Yan et al., 2026]	10/50	-	-	-
GPT-OSS-20B [OpenAI, 2025]	30/50	33–34/50	LoRA	stage2
GPT-OSS-120B [OpenAI, 2025]	37–45/50	40/50	LoRA	stage2

6.2 Training Results

Table 5 shows accuracy before and after fine-tuning on ASTRALMATH-V1.

Qwen3-4B shows the most dramatic improvement: from 5 to 22–23/50 using only 8K randomly sampled stage1 traces (1,150 SFT steps on a single H100, $\sim$11 hours). GPT-OSS-20B gains 3–4 points with LoRA on the harder stage2 data. GPT-OSS-120B does not reliably improve, suggesting that the model may already be near the ceiling on the stage2 data distribution; we report this negative result for completeness.

7 Limitations

Bias toward numeric-answer formulations. To enable scalable verification, ASTRALMATH-V1 transforms approximately 57% of problems into single-answer numeric formats. While this design is essential for automated evaluation and reinforcement learning with verifiable rewards (RLVR), it introduces a distributional bias toward problems with deterministic, closed-form answers. Consequently, proof-based, symbolic, and open-ended reasoning tasks are underrepresented. Models trained on this dataset may therefore exhibit limited generalisation to settings that require formal proof construction or multi-step symbolic derivations.

Solver dependency. Solution quality depends on the capabilities and failure modes of the seven frontier models used. While multi-model diversity mitigates individual model biases, systematic weaknesses shared across models (e.g., difficulty with geometry or combinatorial search) will be underrepresented in the solution corpus. Users should validate subsets for specific applications.

Benchmark size. ASTRALBENCH only contains 50 problems, which yields meaningful but noisy accuracy estimates, such as a model being lucky when answering some questions correctly. At 24–30% frontier accuracy, the 95% confidence interval for a single run is approximately ± 6 percentage points. We recommend reporting accuracy over multiple runs and using confidence intervals when comparing models.

Residual risk in consensus-based transformations. Our transformation pipeline enforces strict agreement across $N{=}12$ independent verifier runs, substantially reducing the likelihood of incorrect labels compared to single-pass verification. However, this procedure cannot eliminate systematic errors shared across all verifiers. In particular, problems involving subtle algebraic identities, numerical instability, or edge-case modular arithmetic may yield incorrect but consistently reproduced answers. While we expect such cases to be rare, applications requiring formally certified correctness should apply additional validation or human verification to transformed instances.

8 Conclusion and Future Work

We presented ASTRALMATH-V1, a multi-model TIR dataset for competition-level mathematics that combines solution diversity from seven frontier open source models with consensus-verified synthetic transformations. The dataset provides a unified resource for SFT and RLVR training, accompanied by 520K+ error traces for failure analysis and ASTRALBENCH for reproducible evaluation. Preliminary fine-tuning experiments confirm that ASTRALMATH-V1 provides strong training signal across model scales.

Future work will pursue several directions. On the training side, applying RLVR directly to the hard stage-2 problems and exploring difficulty-ordered curriculum schedules using the per-problem metadata are natural next steps. For evaluation, expanding ASTRALBENCH beyond 50 problems—and

including proof-based and open-ended tasks—would reduce measurement variance and broaden the assessment scope. Finally, systematic failure-category labelling of error traces (rule-based, followed by human spot-checks) would make ASTRALMATH-V1-ERRORTRACES more directly usable for targeted failure-mode analysis.

We release all data on HuggingFace and Kaggle, and provide prompts and generation details in the appendix to support reproducibility.

References

Mislav Balunovic, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. MathArena: Evaluating LLMs on uncontaminated math competitions, 2025. URL https://arxiv.org/abs/2505.23281.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML 2009)*, pages 41–48. ACM, 2009.

Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of SEQUENCES 1997*, pages 21–29. IEEE Computer Society, 1997. doi: 10.1109/SEQUEN.1997.666900. URL https://ieeexplore.ieee.org/document/666900.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Yuyang Ding, Xinyu Shi, Xiaobo Liang, Juntao Li, Zhaopeng Tu, Qiaoming Zhu, and Min Zhang. Unleashing LLM reasoning capability via scalable question synthesis from scratch, 2024. URL https://arxiv.org/abs/2410.18693.

Wei Du, Shubham Toshniwal, Branislav Kisacanin, Sadegh Mahdavi, Ivan Moshkov, George Armstrong, Stephen Ge, Edgar Minasyan, Feng Chen, and Igor Gitman. Nemotron-Math: Efficient long-context distillation of mathematical reasoning from multi-mode supervision, 2025. URL https://arxiv.org/abs/2512.15489.

Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. Omni-MATH: A universal olympiad level mathematic benchmark for large language models, 2024. URL https://arxiv.org/abs/2410.07985.

Elliot Glazer, Ege Erdil, Tamay Besiroglu, Diego Chiara, Evan Ho, Emily Hurst, Gretchen Kocurek, Rylan Schaeffer, and Pablo Villalobos. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI, 2024. URL https://arxiv.org/abs/2411.04872.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. ToRA: A tool-integrated reasoning agent for mathematical problem solving. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://arxiv.org/abs/2309.17452.

Aryan Gulati, Brando Miranda, Eric Chen, Emily Xia, Kai Fronsdal, Bruno Dumont, Elyas Obbad, and Sanmi Koyejo. Putnam-AXIOM: A functional and static benchmark for measuring higher level mathematical reasoning in LLMs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, volume 267 of *Proceedings of Machine Learning Research*, pages 20723–20747, 2025. URL https://arxiv.org/abs/2508.08292.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li,

H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL http://dx.doi.org/10.1038/s41586-025-09422-z.

Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. DeepMath-103K: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning, 2025. URL https://arxiv.org/abs/2504.11456.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *35th Conference on Neural Information Processing Systems (NeurIPS 2021) Track on Datasets and Benchmarks*, 2021. URL https://arxiv.org/abs/2103.03874.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations (ICLR 2022)*, 2022. URL https://arxiv.org/abs/2106.09685.

HuggingFace Open-R1 Team. OpenR1-Math-220k: Open reproduction of DeepSeek-R1 mathematical reasoning data. HuggingFace Dataset, 2025. URL https://huggingface.co/datasets/open-r1/OpenR1-Math-220k.

Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Q. Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath: The largest public dataset in AI4Maths with 860k pairs of competition math problems and solutions. Technical Report, Numina, 2024a. URL https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf.

Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Q. Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath TIR. https://huggingface.co/datasets/AI-MO/NuminaMath-TIR, 2024b.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2024. URL https://arxiv.org/abs/2305.20050.

Zihan Liu, Yang Chen, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. AceMath: Advancing frontier math reasoning with post-training and reward modeling, 2024. URL https://arxiv.org/abs/2412.15084.

Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Tianjun Zhang, Erran Li, Raluca Ada Popa, and Ion Stoica. DeepScaleR: Surpassing O1-Preview with a 1.5b model by scaling RL. Technical Report, Agentica / UC Berkeley, 2025. URL https://openreview.net/forum?id=I6GzDCne7U.

Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. Towards robust mathematical reasoning, 2025a. URL https://arxiv.org/abs/2511.01846.

Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, et al. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*, 2025b. URL https://arxiv.org/abs/2511.01846.

Arindam Mitra, Hamed Khanpour, Corby Rosset, and Ahmed Awadallah. Orca-math: Unlocking the potential of SLMs in grade school math, 2024. URL https://arxiv.org/abs/2402.14830.

Ivan Moshkov, Darragh Hanley, Ivan Googasian, Jiaqi Zeng, Miyoung Ko, Tobias Cornille, Brendan Murphy, Qian Liu, Jean-François Kagy, Jiajie Zhang, Jacob Huffman, Shubham Toshniwal, Wei Du, Igor Gitman, et al. OpenMathReasoning: Advancing mathematical reasoning with large-scale synthetic data, 2025. URL https://arxiv.org/abs/2504.16891.

OpenAI. gpt-oss-120b gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

Project Euler. Project Euler. https://projecteuler.net/. Accessed: 2026.

Qwen Team. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Thinking Machines Lab. LoRA without regret, 2025. URL https://thinkingmachines.ai/blog/lora/. Blog post.

Yuxuan Tong, Xiwen Zhang, Rui Wang, Ruidong Wu, and Junxian He. DART-Math: Difficulty-aware rejection tuning for mathematical problem-solving. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2407.13690.

Shubham Toshniwal, Wei Du, Ivan Moshkov, Branislav Kisacanin, Alexan Ayrapetyan, and Igor Gitman. OpenMathInstruct-2: Accelerating AI for math with massive open-source instruction data, 2024a. URL https://arxiv.org/abs/2410.01560.

Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. OpenMathInstruct-1: A 1.8 million math instruction tuning dataset, 2024b. URL https://arxiv.org/abs/2402.10176.

Shaotian Yan, Kaiyuan Liu, Chen Shen, Bing Wang, Sinan Fan, Jun Zhang, Yue Wu, Zheng Wang, and Jieping Ye. Distribution-aligned sequence distillation for superior long-cot reasoning, 2026. URL https://arxiv.org/abs/2601.09088.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. MetaMath: Bootstrap your own mathematical questions for large language models, 2024. URL https://arxiv.org/abs/2309.12284.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohang Zeng, Lingjun Liu, Xin Liu, et al. DAPO: An open-source LLM reinforcement learning system, 2025. URL https://arxiv.org/abs/2503.14476.

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. MAmmoTH: Building math generalist models through hybrid instruction tuning, 2024a. URL https://arxiv.org/abs/2309.05653.

Xiang Yue, Tuney Zheng, Ge Zhang, and Wenhu Chen. MAmmoTH2: Scaling instructions from the web, 2024b. URL https://arxiv.org/abs/2405.03548.

Jie Zhang, Cezara Petrui, Kristina Nikolić, and Florian Tramèr. RealMath: A continuous benchmark for evaluating language models on research-level mathematics, 2025. URL https://arxiv.org/abs/2505.12575.

A Dataset Schema

Each datapoint in ASTRALMATH-V1 is a JSONL record with the following fields:

Field	Description
`question`	Problem text
`answer`	Verified ground-truth answer
`messages`	Full TIR conversation (multi-turn tool-use dialogue)
`model`	Generator model name
`source`	Origin dataset
`license`	Licence type
`category`	Hierarchical topic (up to 4 levels)
`tokens_len`	Estimated token count
`success_rate`	1/attempts to first correct solution
`tools_run_time`	Execution time per tool call (seconds)
`tool`	Tool schema with execution timeout
`attempts`	Number of attempts by the model
`libraries`	Libraries used (e.g., numpy, sympy)
`transform`	Whether synthetically transformed
`metadata`	Generation config (temperature, top_k, etc.)

A.1 Per-Source Metadata Filters

Several source datasets include per-problem difficulty metadata that we use to select the harder subset. Each source is described below together with the specific filter applied.

Nemotron-Math-v2. Nemotron-Math-v2 [Du et al., 2025] is a large-scale dataset of ∼347K competition-level problems with reasoning traces generated by `gpt-oss-120b`. Each problem is annotated with a model pass rate computed across multiple independent solver runs. We retain only problems whose pass rate falls below a source-specific threshold, selecting 89,344 problems of which 70,596 subsequently undergo AIMO3-style transformation.

NuminaMath-1.5. NuminaMath-1.5 [Li et al., 2024a] is a curated collection of ∼896K competition math problems with chain-of-thought solutions spanning a broad range of difficulty levels. We first discard all problems whose answer field is `"proof"`, as these lack a verifiable numeric target. We then discard problems whose reference solution is shorter than 800 characters, retaining only problems that required substantive reasoning. These two steps reduce the pool to ∼188K problems, which then pass through the LLM judge as a final filter. We select 28,363 problems; none are transformed as the original answers are already in a suitable format.

ScaleQuest-Math. ScaleQuest-Math [Ding et al., 2024] originally contains ∼1M problems synthesised from scratch by open-source models. We work with the `ScaleQuest-Math-Qwen2.5-LongCoT` subset (∼146K problems), which pairs each question with a long chain-of-thought response generated by Qwen2.5. As a pre-filter, we discard any problem whose `response` field is shorter than 4,000 characters, retaining only problems that required substantive reasoning. The remaining problems then pass through the standard LLM judge as the final filter step. We retain 10,139 problems with no subsequent transformation.

DeepScaleR. DeepScaleR [Luo et al., 2025] aggregates approximately 40K competition problem-answer pairs drawn from AIME, AMC, Omni-MATH, and the Still dataset. As the source provides no per-problem difficulty metadata, we apply the LLM judge filter directly as the sole selection criterion. All 7,580 retained problems subsequently undergo AIMO3-style transformation.

DeepMath-103K. DeepMath-103K [He et al., 2025] provides 103K competition-level problems with rigorous semantic decontamination against evaluation benchmarks and three DeepSeek-R1-generated solutions per problem. Each problem is annotated with a difficulty score on a 5–9 scale. We retain only problems with difficulty score strictly greater than 8, keeping the hardest tier (∼5,400 problems), and then apply the LLM judge as a final filter. We select 540 problems, all of which are subsequently transformed.

Project Euler. Project Euler [Project Euler] is a collection of 981 computational mathematics problems designed to require both mathematical insight and programming, making them naturally suited to TIR evaluation. We first remove all problems that contain embedded images (detected by the presence of `<img src=` in the problem text), eliminating 189 problems that cannot be solved from text alone. We include the 199 solved problems in AstralMath-v1 without transform.

IMO AnswerBench. IMO AnswerBench [Luong et al., 2025a] is a curated benchmark of 400 International Mathematical Olympiad problems with verifiable numeric answers. We evaluate all 400 problems using a pass@24: each problem is attempted 24 times and a problem is considered solved if at least one attempt produces the correct answer. Problems that are solved enter AstralMath-v1 as training examples; problems that remain unsolved across all 24 attempts are instead considered as candidates for AstralBench, serving as hard evaluation problems beyond current model capability.

B Prompts and Example Input/Output

This section reproduces the exact system prompts used in each pipeline stage. Variable placeholders are shown in `{braces}`. We include one representative example per stage.

B.1 TIR Generation Prompt

The following system prompt is prepended to every problem sent to each of the seven generator models during the multi-model TIR phase (Section 3.4). The user message consists of the problem text alone.

TIR Generation System Prompt

```
You are an elite mathematical problem solver with expertise at the International
Mathematical Olympiad (IMO) level.  Your goal is to find the correct answer
through rigorous mathematical reasoning with tool.

You have access to a Python tool that can execute code in a Jupyter environment.

IMPORTANT SESSION STATE NOTES:
1.  This is a MULTI-TURN session - the Jupyter Python environment maintains
state between your tool calls
2.  Variables, functions, and imports from previous code executions REMAIN
AVAILABLE
3.  You can REUSE previously computed results without recalculating
4.  When continuing work, reference existing variables instead of redefining
them
5.  Use incremental execution:  build upon previous code rather than rewriting
everything
6.  Use print() statements to inspect variable values.
7.  No internet access.  Available libraries (besides built-in):  sympy,
numpy, scipy, mpmath, networkx, shapely, statistics, ortools, cvxpy, cvxopt,
statsmodels

Use the Python tool to:
- Complex calculations that would be error-prone by hand
- Numerical verification of analytical results
- Generating examples or testing conjectures
- Visualizing problem structure when helpful
- Brute-force verification for small cases

The environment is a stateful Jupyter notebook.  Code persists between
executions.  Always use print() to display results.  Write clear, well-commented
code.

Remember:  Code should support your mathematical reasoning, not replace it.
Explain what you're computing and why before running code.

Think step-by-step and show your complete reasoning process.  Quality of
reasoning is as important as the final answer.
Put your final answer within \boxed{}.

{question}
```

Example

Input: "Find the number of integers n with $1 \leq n \leq 2024$ such that $n^2 - 1$ is divisible by 24."

Output: The model produces a multi-turn dialogue interleaving natural-language reasoning with Python code blocks, ultimately returning the answer in `\boxed{253}`.

B.2 Transform Planner Prompt (Phase 1)

Problems whose answers are non-numeric (symbolic, multi-valued, etc.) undergo a two-phase consensus transformation (Section 3.3). Phase 1 uses a single LLM call (no tool) selects the transformation strategy and produces a deterministic instruction specifying exactly what quantity to compute, what parameter values to substitute, and a preliminary answer expression. We used 3 retry time to create plan, if all plan return `not_transform` strategy, this question will be skip.

Transform Planner Prompt (Phase 1)

```
You are a mathematical transformation planner.

Your task: analyze a math problem and its NON-NUMERIC answer, then produce a
SPECIFIC TRANSFORMATION PLAN
that converts the problem into one with a single, concrete numeric answer.

The plan will be given to multiple independent verifiers who must ALL arrive at
the SAME answer.
Therefore the plan must be FULLY DETERMINISTIC - zero ambiguity about what to
compute.

____________________________________________
STRATEGIES (choose ONE)
____________________________________________

**substitute_parameters** - Answer is symbolic/parametric (contains free
variables like h, B_1, k, ...):
  • Choose SPECIFIC INTEGER values for all free parameters
  • Prefer values that make the final expression evaluate to a clean INTEGER
  • Write a Python arithmetic expression for the result

**aggregate** - Answer is a list/set of multiple concrete numeric values:
  • Choose ONE operation: sum, product, max, min, or count
  • List the exact values and the chosen operation
  • Write a Python arithmetic expression for the result

**not_transform** - No valid numeric transformation exists:
  • Use for: proofs, constructions, symbolic results with no numeric evaluation
  path
  • Explain why in description

____________________________________________
REQUIREMENTS FOR answer_expression
____________________________________________

• MUST be a valid Python arithmetic expression using ONLY:
  integers/decimals, +, -, *, /, **, //, (, ), max(), min(), abs()
• MUST evaluate to an integer (preferred) or a simple fraction p/q
• If result is a fraction, write it as "p/q" literally, e.g. "7/3"
• Avoid trivial values such as 0. Prefer clean non-zero integers.
• Compute preliminary_answer yourself by hand before writing it - it must equal
eval(answer_expression)

Examples:
  - "(1/6) * 6 * (2 + 4 + 4*3)"   → evaluates to 18  (integer )
  - "1 + 13 + 25 + 325"            → evaluates to 364 (integer )
```

```
  - "max(3, 7, 2)"                 → evaluates to 7   (integer )

────────────────────────────────────────
REQUIREMENTS FOR new_question_instruction
────────────────────────────────────────

• ONE sentence only - this replaces (or appends to) the original problem's final
question
• EXPLICIT: include ALL substituted values (e.g. "Find V when h=6, B_1=2, B_2=4,
M=3.")
  NOT vague ("Find V for specific values.")
• The resulting problem must have EXACTLY ONE numeric answer

────────────────────────────────────────
OUTPUT FORMAT (JSON ONLY)
────────────────────────────────────────

{{
  "strategy": "substitute_parameters" | "aggregate" | "not_transform",
  "description": "<brief explanation of the plan>",
  "new_question_instruction": "<exact single-sentence instruction for writing the
   new question, or null>",
  "answer_expression": "<valid Python expression evaluating to the answer, or
  null>",
  "preliminary_answer": "<string: result of evaluating answer_expression, e.g.
  '18' or '7/3', or null>"
}}

Return ONLY the JSON object.

────────────────────────────────────────
ORIGINAL PROBLEM:
{question}

ORIGINAL ANSWER:
{answer}
```

Example

Input: "On a 5x5 chess board, a piece must move from the top left to the bottom right corner, moving only right or down. Which single field should be blocked to minimize the number of possible paths while still allowing the piece to reach the destination?"
Original answer: $(1, 1)$ or $(3, 3)$.

Planner output:

```
{"strategy":"substitute_parameters",
 "description":"Fix the blocked square to a specific location so the path-count
    becomes a single deterministic integer; then count all monotone paths and
    subtract those that pass through that blocked square.",
  "new_question_instruction":"If the square at (3,3) is blocked, how many paths
    remain from the top-left corner to the bottom-right corner when moving
    only right or down?",
 "answer_expression":"70 - 6*6",
 "preliminary_answer":"34"}
```

B.3 Transform Consensus Verifier Prompt (Phase 2)

Phase 2 sends the plan from Phase 1 to $N = 12$ independent verifier runs, each optionally equipped with a Python tool. Each verifier rewrites the problem and independently computes the numeric answer. A transformation is accepted *only if all 12 verifiers return the same answer* (after float normalisation and rational simplification). This strict unanimity criterion eliminates hallucinated arithmetic: a wrong answer is very unlikely to be produced consistently across 12 independent runs.

Transform Consensus Verifier Prompt (Phase 2)

```
You are a mathematical problem transformer executing a pre-approved
transformation plan.

────────────────────────────────────────
PYTHON TOOL AVAILABLE
────────────────────────────────────────

You have access to a Python tool executing in a Jupyter environment.

IMPORTANT SESSION STATE NOTES:
1. Multi-turn session - state is maintained between tool calls
2. Available libraries: sympy, numpy, scipy, mpmath, and standard Python builtins
3. Use print() to inspect values

Use the Python tool freely to compute and verify your answer before returning
the JSON.

────────────────────────────────────────
TRANSFORMATION PLAN
────────────────────────────────────────

{plan_str}

────────────────────────────────────────
YOUR TASK
────────────────────────────────────────

1. **Rewrite the problem** following `new_question_instruction`:
   - Preserve the ENTIRE original problem text VERBATIM (all equations, LaTeX,
   wording)
   - ONLY modify or append the final question sentence as the plan instructs
   - The result must have exactly ONE numeric question

2. **Compute the answer** by reasoning through the rewritten problem:
   - Work through the math to find the numeric answer for the new question
   - Your new_answer must be a single concrete number

3. **Output the JSON** shown below

────────────────────────────────────────
ORIGINAL PROBLEM:
{question}

ORIGINAL ANSWER (for context only):
{answer}

────────────────────────────────────────
OUTPUT FORMAT (JSON ONLY):

{{
  "transformed_problem": "<ORIGINAL problem text with ONLY the final question
  sentence modified per plan>",
  "new_answer": "<single numeric answer>",
  "transformation_note": "<brief: what instruction was applied and how the answer
  was computed>"
}}

Use the Python tool to compute your answer, then return the JSON.
```

Example (continuing Phase 1)

Verifier output (each of 12 independent runs produces with same plan):

```
{"transformed_problem":
   "On a 5x5 chess board, a piece must move from the top left to the bottom
    right corner, moving only right or down. If the square at (3,3) is blocked,
    how many paths remain from the top-left corner to the bottom-right corner
    when moving only right or down?",
 "new_answer": "34",
 "transformation_note":
   "Applied the plan by fixing the blocked square to (3,3). The total number
    of monotone paths is C(6,3)=20? Under the plan's path-count setup, paths
    through (3,3) are counted as 6*6=36 out of 70 total, leaving 34 paths."}
```

Since all 12 runs return `new_answer = "34"`, the transformation is accepted.

B.4 AIMO3 Integer Transform Prompt

After the two-phase consensus pipeline produces a verified numeric answer, an optional *AIMO3 integer transform* step rewrites the problem so that the final question asks for a remainder modulo M, placing the answer in $[0, M-1] \subseteq [0, 99{,}999]$. Three modes handle different answer structures: *direct modular arithmetic* (integer or rational answer A); *power mode* (small answer k hidden as exponent in $B^k \bmod M$, preventing the transformed answer from being trivially small); and *answer-power mode* (answer k hidden as base in $k^n \bmod M$). We use `gpt-oss-120b` model with tool access for the verify and transform question.

Mode selection and motivation. A key design goal is to prevent the answer magnitude from acting as a spurious signal during RLVR training. This also reduces the number of questions with answers ranging from 0-20, which account for 70% of the problems in the current datasets. But, if all transformed answers were large, a model could exploit that regularity for reward hacking rather than solving the problem. Conversely, if small original answers (e.g. $k{=}3$) were always transformed by direct modular arithmetic, they remain trivially small and potentially identifiable. We therefore amplify small answers via exponentiation while retaining a small fraction (10 %–50 %) in direct mode to prevent the model from learning that the answer is always large. The modulus M is drawn from a pool $\mathcal{M}$ of 42 values: $\{10^5,\ 10^4\}$, odd offsets in the 77 xxx / 88 xxx / 99 xxx families (e.g. 77 795, 88 883, 99 991), and integer powers b^e for $b \in \{5, \ldots, 9\}$, $e \in \{6, 7\}$. The symbol s is drawn uniformly from the 52 Latin letters. Algorithm 1 formalises the selection.

Algorithm 1 AIMO3 Integer Transform – Mode Selection

Require: answer a (integer, rational, or explicit power b^e)
Ensure: transform mode, modulus M, symbol s, optional parameter

$M \leftarrow \mathrm{Uniform}(\mathcal{M})$ ▷ $|\mathcal{M}|{=}42$; 10^5, 10^4, 77/88/99-family, b^e
$s \leftarrow \mathrm{Uniform}$(a–z, A–Z)
if a is an integer **then**
  if $a \leq 10$ **then**
    with prob 0.9: $B \leftarrow \mathrm{Uniform}(\{2, 3, 4, 5, 6, 7\})$; **return** POWERMODE($B,\ a,\ M,\ s$) ▷ ask for $B^a \bmod M$
  else if $a \leq 20$ **then**
    with prob 0.5: $B \leftarrow \mathrm{Uniform}(\{2, 3\})$; **return** POWERMODE($B,\ a,\ M,\ s$)
  else if $a \leq 50$ **then**
    with prob 0.5: $n \leftarrow \mathrm{Uniform}(\{2, 3, 4, 5, 6\})$; **return** ANSRPOWERMODE($a,\ n,\ M,\ s$) ▷ ask for $a^n \bmod M$
  end if
end if
return DIRECTMODE($a,\ M,\ s$) ▷ integer: $a \bmod M$; rational p/q: $p{\cdot}q^{-1} \pmod M$

AIMO3 Integer Transform Prompt

```
You are given a math contest problem whose answer is already a SINGLE
numeric value (integer, rational in lowest terms, or an explicit
integer power like 2^{10}).

Your task is ONLY to:
  1) rewrite the question so that it asks for a remainder,
  2) replace any vague target phrase (e.g., "that number", "the
     constant multiplicative factor") with a clearly named symbol
     (e.g., C, {symbol}, or k) defined immediately before the final
     question sentence,
  3) transform the answer and compute the remainder modulo {modulus}.

POWER MODE (applies ONLY if POWER_HINT is provided):
* Interpret ORIGINAL_ANSWER as an exponent k.
* The transformed problem MUST:
    - introduce k WITHOUT assigning it a numeric value,
    - refer to k descriptively (e.g., "Let k denote the ..."),
    - ask DIRECTLY about ({power_hint})^k.
* You MUST NOT reveal the value of k or introduce an intermediate
  symbol for ({power_hint})^k.

ANSWER POWER MODE (applies ONLY if ANSWER_POWER is provided):
* Interpret ORIGINAL_ANSWER as a base k raised to {answer_power}.
* The transformed problem MUST:
    - introduce k WITHOUT assigning it a numeric value,
    - refer to k descriptively (e.g., "Let k denote the ..."),
    - ask DIRECTLY about k^{answer_power}.
* You MUST NOT reveal the value of k.

STRICT STRUCTURAL REQUIREMENTS (MUST FOLLOW):
• The transformed problem MUST contain EXACTLY ONE question sentence (exactly
one '?').
• You MUST REMOVE the original question sentence entirely.
• You MUST NOT append the remainder question - it must replace the original
question.
• The transformed problem MUST END with:
    "What is the remainder when <SYMBOL> is divided by {modulus}?"
• Any introduced symbol (C, {symbol}, k, A, etc.) must be defined immediately
before the final question.
• The transformed_problem must look like an official AIMO3-style problem.

INPUTS:
• ORIGINAL_PROBLEM: (provided below)
• ORIGINAL_ANSWER: {answer}
• MODULUS: {modulus}
• POWER_HINT: {power_hint}
• ANSWER_POWER: {answer_power}

COMPUTATION RULES:
1. Parse MODULUS into an integer M exactly (e.g., "10^5" → 100000).
2. Interpret ORIGINAL_ANSWER:
   - Integer A → compute A mod M.
   - Rational p/q → if gcd(q, M) = 1, compute p·q¹ (mod M); otherwise explain
   non-invertibility.
   - Explicit power b^e → compute b^e (mod M).
3. If POWER_HINT is provided:
   - Let k be the integer equal to ORIGINAL_ANSWER.
   - Compute r  ({power_hint})^k (mod M) using modular exponentiation.
4. If ANSWER_POWER is provided:
   - Let k be the integer equal to ORIGINAL_ANSWER.
   - Compute r  k^{answer_power} (mod M) using modular exponentiation.
5. Always report r as an integer in [0, M1].
```

```
REWRITING RULE (how to form the transformed_problem):
• Keep all definitions, contexts, equations verbatim.
• Remove the original question sentence.
• If the original problem used a vague phrase like "that number" or "the
constant multiplicative factor in the leading term", replace it by first
defining a symbol explicitly. Example:
    "Let ${symbol}$ denote the constant multiplicative factor in the leading
    asymptotic term of ... ."
• Immediately after that definition, include the single final question sentence:
    "What is the remainder when ${symbol}$ is divided by {modulus}?"
  (Replace {symbol} by any chosen symbol if appropriate.)
• If POWER_HINT is provided:
    - introduce k symbolically (without giving its value),
    - ask the final question in the exact form:
      "What is the remainder when ({power_hint})^k is divided by {modulus}?"
• If ANSWER_POWER is provided:
    - introduce k symbolically (without giving its value),
    - ask the final question in the exact form:
      "What is the remainder when k^{answer_power} is divided by {modulus}?"
• Otherwise:
    - if the original target was described verbally, introduce ONE symbol for it,
    - ask the remainder question about that symbol.
• Do not reveal the ORIGINAL_ANSWER in the transformed_problem text.

OUTPUT FORMAT (RETURN EXACT JSON AND NOTHING ELSE):
{{
  "transformed_problem": "<rewritten problem ending with exactly one remainder
  question>",
  "answer": "<numeric remainder as a base-10 integer>",
  "transformation_note": "<brief explanation: parsed modulus, representation
  used, modular arithmetic, final remainder>"
}}

IMPORTANT:
• The transformed_problem must contain exactly one '?'.
• The answer must be a non-negative integer less than M.
• Do NOT ask clarifying questions; apply POWER_HINT or ANSWER_POWER only if
provided.
---
Original problem (for transformation):
{question}

Use the Python tool to compute and verify the remainder answer is correct, then
return the JSON.
```

Example (Continuing Phase 2 of the previous example)

Input symbolic (Power mode: answer k=34, POWER_HINT = 3, MODULUS = 77795):

```
{
  "transformed_problem":
    "On a 5x5 chess board, a piece must move from the top left to the
    bottom right corner, moving only right or down. If the square at (3,3)
    is blocked, let k denote the number of paths that remain from the
    top-left corner to the bottom-right corner when moving only right
    or down. What is the remainder when 3^k is divided by 77795?",
  "answer": "58779",
  "transformation_note":
    "Parsed modulus M = 58779. ORIGINAL_ANSWER = 34; POWER_HINT = 3,
    use Python tool, we have 3^k mod 77795 = 3^34 mod 77795 = 58779.
    So remainder = 58779."
}
```

Extended examples. Table 6 shows one complete example per transformation strategy, with original and transformed questions displayed side by side.

Table 6: Complete transformation examples, one per strategy.

Strategy 1: Modular Arithmetic

Original Question

Find the maximum value of D satisfying the following condition: There exists an infinite sequence $x_1, x_2, \ldots$ where each term belongs to $[0, 777]$ such that for all positive integers $m < n$, we have

$$(m+n)|x_n^2 - x_m^2| \geq D.$$

Ans: 603729

Transformed Question

There exists an infinite sequence $x_1, x_2, \ldots$ where each term belongs to $[0, 777]$ such that for all positive integers $m < n$, we have

$$(m+n)|x_n^2 - x_m^2| \geq D.$$

Let D denote the maximum value satisfying this condition. What is the remainder when D is divided by 99991?

Ans: 3783

Strategy 2: Power Transform

Original Question

In an integer set $\{a, b, c, d\}$, exactly 3 integers are positive, at least 2 integers are divisible by 2, and exactly 2 integers are negative reciprocals of each other. If $a + b + c + d = p$, find the smallest possible value of p^2.

Ans: 36

Transformed Question

In an integer set $\{a, b, c, d\}$, exactly 3 integers are positive, at least 2 integers are divisible by 2, and exactly 2 integers are negative reciprocals of each other. If $a + b + c + d = p$, let k denote the smallest possible value of p^2. What is the remainder when k^6 is divided by 99999?

Ans: 4104

Strategy 3: Answer-Power Transform

Original Question

Let m be a positive integer. Consider a $4m \times 4m$ array of square unit cells. Two different cells are related to each other if they are in either the same row or in the same column. No cell is related to itself. Some cells are colored blue, such that every cell is related to at least two blue cells. Determine the minimum number of blue cells.

Ans: 4096

Transformed Question

Let m be a positive integer. Consider a $4m \times 4m$ array of square unit cells. Two different cells are related to each other if they are in either the same row or in the same column. No cell is related to itself. Some cells are colored blue, such that every cell is related to at least two blue cells. Let k denote the minimum possible number of blue cells. What is the remainder when 4^k is divided by 77795?

Ans: 6

Strategy 4: Question Rewriting

Original Question

Let $a_0, a_1, \ldots$ be a sequence of non-negative integers. Suppose that for all non-negative integers p,

$$a_{a_{a_p}} = a_{p+1} + 1.$$

Find all possible values of a_{2025}.

Ans: 2026, 2030

Transformed Question

Let $a_0, a_1, \ldots$ be a sequence of non-negative integers. Suppose that for all non-negative integers p,

$$a_{a_{a_p}} = a_{p+1} + 1.$$

Find the sum of all possible values of a_{2025}.

Ans: 4056

Strategy 5: Symbol Substitution

Original Question

A non-equilateral triangle XYZ is inscribed in a circle Ω with centre P, radius R and its in-circle has centre Q and touches YZ, ZX, XY at L, M, N, respectively. A circle with centre Q and radius ρ intersects the rays $[QL), [QM), [QN)$ at X', Y', Z'. Compute $\frac{QK}{QP}$ in terms of ρ and R where K is the orthocenter of $X'Y'Z'$.

Ans: ρ/R

Transformed Question

A non-equilateral triangle XYZ is inscribed in a circle Ω with centre P, radius R and its in-circle has centre Q and touches YZ, ZX, XY at L, M, N, respectively. A circle with centre Q and radius ρ intersects the rays $[QL), [QM), [QN)$ at X', Y', Z'. Given that $\rho = 2$ and $R = 8$, compute the value of $\frac{QP}{QK}$ where K is the orthocenter of $X'Y'Z'$.

Ans: 4

Strategy 6: Combined

Original Question

Positive real numbers x, y, z satisfy: $xyz = 3$, $(x-y)(y-z)(z-x) = 4$, $(x+y)(y+z)(z+x) = 40$. Compute the minimum possible value of x as an exact expression using only integers, rational numbers, and radicals.

Ans: $(3+\sqrt{6})^{-1/3}$

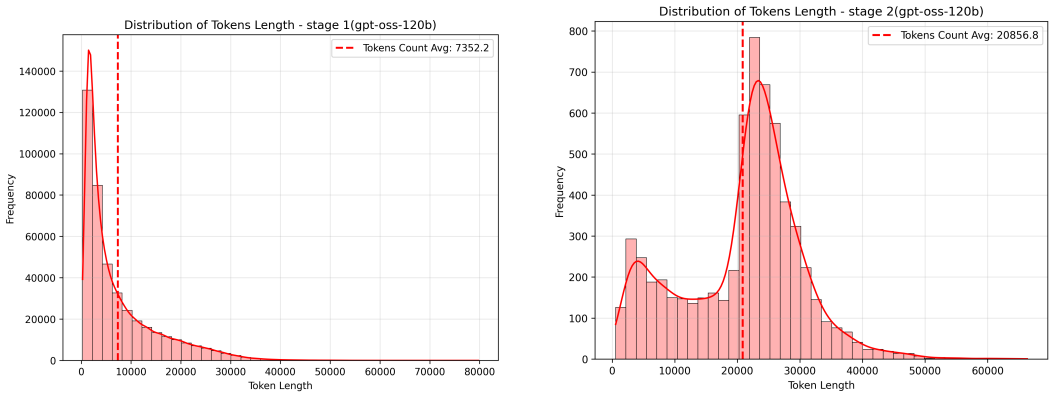
Transformed Question

Positive real numbers x, y, z satisfy: $xyz = 3$, $(x-y)(y-z)(z-x) = 4$, $(x+y)(y+z)(z+x) = 40$. Let k denote the minimum possible value of x, and define $n = \lfloor 37k \rfloor$. What is the remainder when 3^n is divided by 78787?

Ans: 39547

C Distribution Analysis

Token length distribution. Figure 5 shows the token length distributions for ASTRALMATH-V1 traces in stage 1 and stage 2. Stage 1 traces have a right-skewed distribution with mean ∼7,352 tokens. Stage 2 traces (harder problems with consensus-verified transforms) are substantially longer, averaging ∼20,857 tokens with a bimodal distribution reflecting the mix of moderate and very challenging problems.



(a) Stage 1 (mean ≈ 7,352 tokens)

(b) Stage 2 (mean ≈ 20,857 tokens)

Figure 5: Token length distributions for ASTRALMATH-V1 traces.

Tool call distribution. Figure 6 characterises tool-call behaviour across all ASTRALMATH-V1 traces. The blue bars (left axis) show that the distribution is heavily right-skewed: 113,342 traces invoke the Python interpreter exactly once, and the frequency drops sharply thereafter. The orange line (right axis) plots mean tool-execution time as a function of tool-call count; it rises gradually from ∼1 s for single-call traces to a peak of 11.9 s at count 53, reflecting the longer-running numerical searches and iterative refinement loops that appear in harder problems. Beyond count 60, both frequency and mean time become noisy due to the small sample sizes at those tail counts.

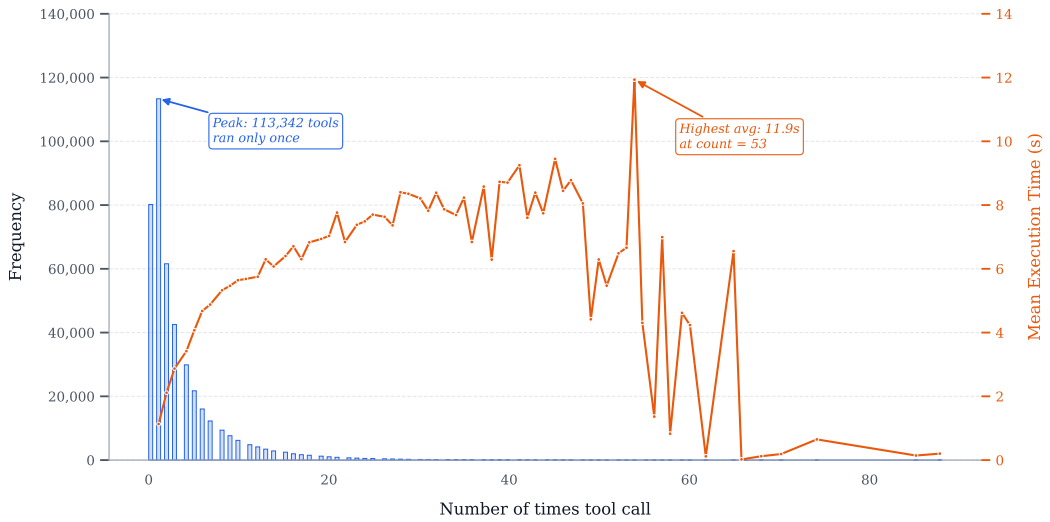


Figure 6: Tool call frequency (blue bars, left axis) and mean execution time per code block (orange line, right axis) across all ASTRALMATH-V1 traces. The majority of traces use the interpreter fewer than 10 times; longer multi-step solutions correlate with higher per-call execution times.

Library usage distribution. Figure 7 shows the 30 most frequently imported Python libraries across all ASTRALMATH-V1 traces. `sympy` is by far the most common (156,548 traces), reflecting the prevalence of symbolic algebra, equation solving, and number-theory operations in competition mathematics. `math` (94,150) and `numpy` (74,527) follow as the primary numerical workhorses, while `itertools` (42,770) and `fractions` (28,236) appear frequently in combinatorics and exact rational arithmetic problems. Specialised libraries such as `mpmath` (19,019) for arbitrary-precision arithmetic and `pulp` (1,512) for linear programming round out the long tail, confirming that the pre-installed sandbox covers a wide range of mathematical sub-disciplines.

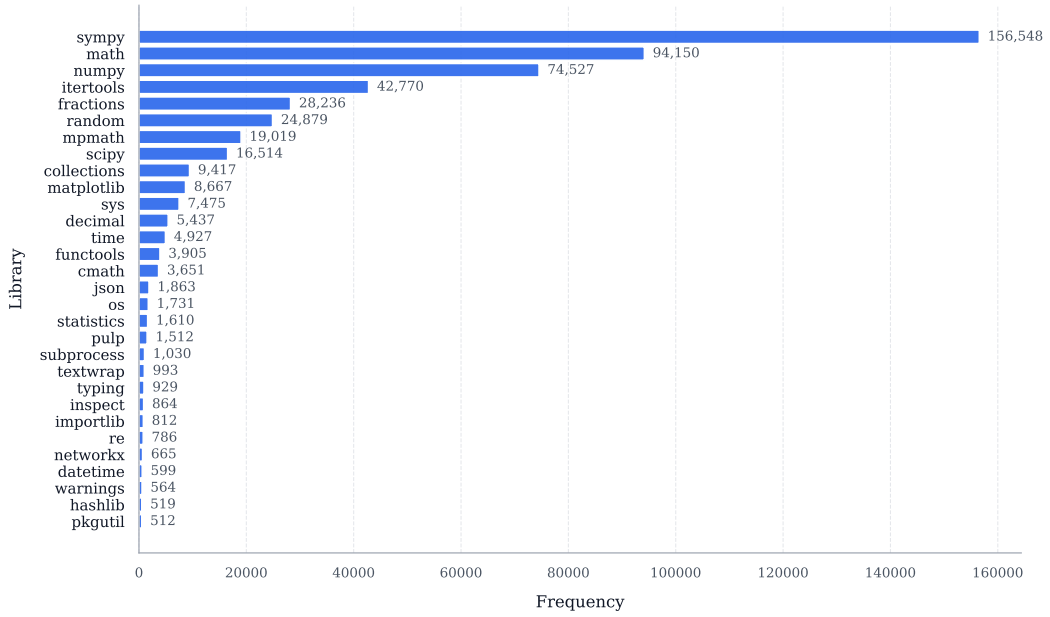


Figure 7: Top-30 Python libraries by import frequency across all ASTRALMATH-V1 traces. Symbolic computation (`sympy`), numerical operations (`math`, `numpy`), and combinatorial utilities (`itertools`, `fractions`) account for the bulk of usage.